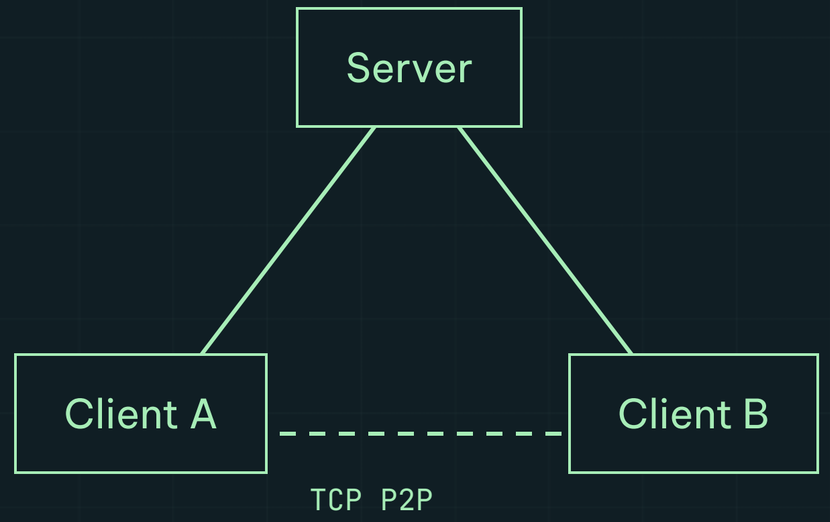
# ChatApp

Kết nối qua tin nhắn.
Chia sẻ tệp trực tiếp.

Giảng viên hướng dẫn: ThS. Nguyễn Lê Văn

Trần Gia Huy · Trần Sỷ Sơn Lâm
Đặng Ngọc Huy

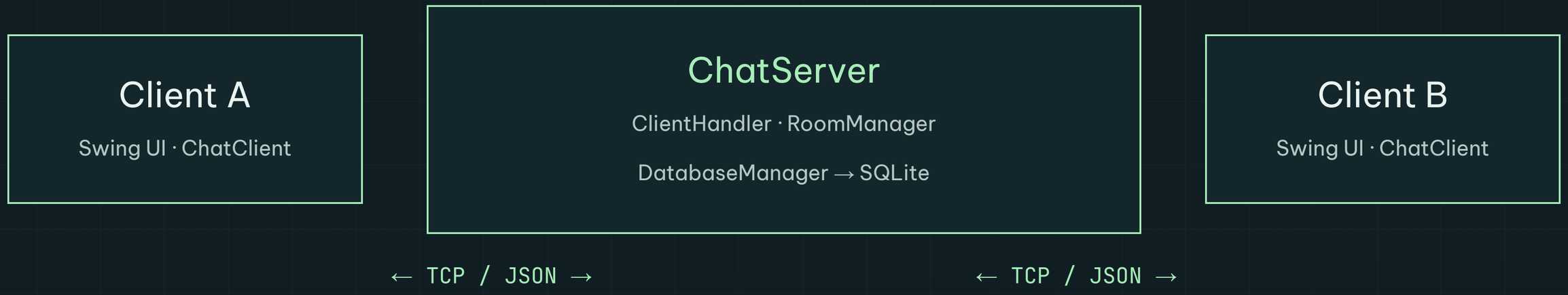


# Thành viên thực hiện

| STT | HỌ VÀ TÊN | MÃ SỐ SINH VIÊN |
|---|---|---|
| 01 | Trần Gia Huy | 2280601246 |
| 02 | Trần Sỷ Sơn Lâm | 2280601697 |
| 03 | Đặng Ngọc Huy | 2380600794 |

Giảng viên hướng dẫn: **ThS. Nguyễn Lê Văn**

# Máy chủ điều phối. Máy khách trao đổi.




Mục tiêu: chat riêng, chat nhóm và chia sẻ file. Tin nhắn/báo hiệu qua server; byte file đi trực tiếp giữa các client.

# Java làm nền. TCP kết nối.

SOURCE / TARGET 11

## Swing/AWT · FlatLaf 3.4.1

Source/target 11; runtime đã xác minh: JDK 21.0.11.

TCP · JAVA.NET

## Gson 2.10.1

Socket / ServerSocket blocking I/O; ExecutorService cached pool.

LƯU TRỮ

## sqlite-jdbc 3.45.1.0

Phiên bản driver JDBC; không phải phiên bản SQLite engine.

**Tuần tự hóa và khung truyền:** Gson chuyển Packet thành JSON; mỗi dòng TCP được bọc AES-128-GCM/Base64. Thân tin nhắn mới có AES-256-GCM riêng (JCA/JCE).

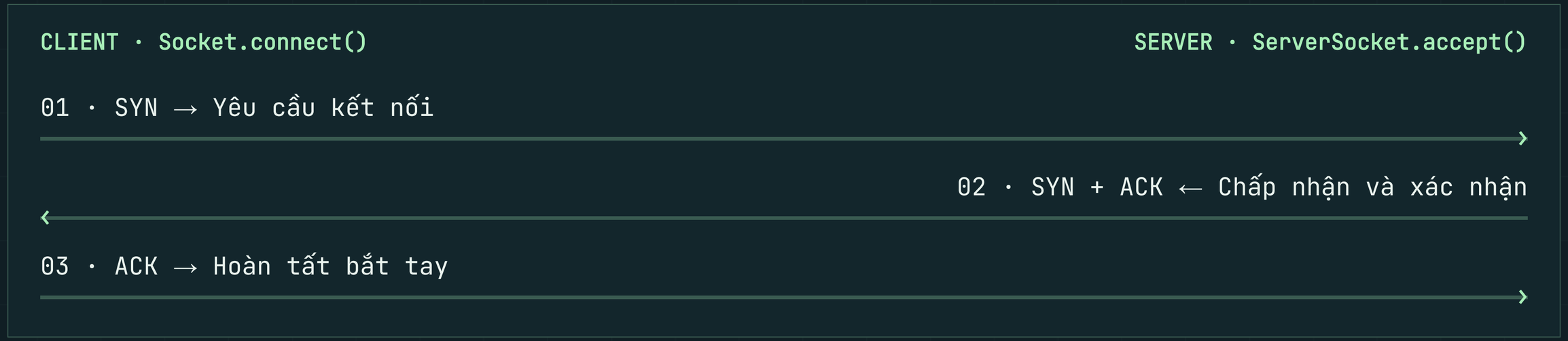
**P2P:** socket TCP riêng, khối 64 KiB, tiếp tục theo byte offset và kiểm tra SHA-256. Mật khẩu dùng SHA-256 + salt 16 byte.

```
Chat body AES-256-GCM → Packet JSON → AES-128-GCM/Base64 → TCP newline
```

**Build/test:** Maven · Compiler 3.11.0 · JAR 3.3.0 · Exec 3.1.0 · JUnit Jupiter 5.10.2 (test scope). JAR không gói dependencies.

# TCP bắt đầu bằng một kết nối.

TCP (Transmission Control Protocol) truyền luồng byte hai chiều giữa hai đầu cuối IP:cổng.




> Kết nối sẵn sàng → hai bên gửi và nhận đồng thời (full-duplex).

Hệ điều hành thực hiện bắt tay TCP. Đăng nhập, Packet và xác nhận đã đọc do ChatApp xử lý ở tầng ứng dụng.

# TCP giữ thứ tự byte, không chia tin nhắn.

TIN CẬY CÓ ĐIỀU KIỆN

### Thứ tự & truyền lại

Số thứ tự và ACK giúp phát hiện byte thiếu, truyền lại và loại trùng. Kết nối vẫn có thể lỗi hoặc timeout.

LUỒNG BYTE

### Không có ranh giới tin

Một lần write() có thể được đọc thành nhiều phần; nhiều lần write() có thể gộp trong một lần read().

ĐÁNH ĐỔI

### Chờ byte bị thiếu

TCP điều khiển luồng và tắc nghẽn. Khi mất đoạn, dữ liệu sau phải chờ đoạn thiếu trong cùng luồng.

```
Chat: Base64(AES-GCM(Packet JSON)) + newline
File: độ dài khối + byte mã hóa · TCP P2P riêng
```

TCP ACK chỉ xác nhận ở tầng TCP; không chứng minh người nhận đã xem tin hoặc đã lưu file. TCP cũng không tự mã hóa dữ liệu.

# TCP hay UDP: chọn theo điều cần giữ.

| Tiêu chí | TCP · luồng byte | UDP · datagram |
|---|---|---|
| **Kết nối** | Thiết lập kết nối; có trạng thái. | Không có bắt tay kết nối ở tầng UDP. |
| **Độ tin cậy** | Truyền lại, loại trùng, giao đúng thứ tự. | Không tự bảo đảm tới nơi, thứ tự hay loại trùng. |
| **Ranh giới dữ liệu** | Ứng dụng phải tự đóng khung bản tin. | Giữ ranh giới từng datagram. |
| **Độ trễ / chi phí** | Có bắt tay; truyền lại có thể gây chờ. | Ít cơ chế hơn; ứng dụng tự lo độ tin cậy nếu cần. |
| **Ví dụ phù hợp** | Chat văn bản, truyền file cần đủ byte. | Thoại/video trực tiếp, trạng thái game; tùy thiết kế. |

> ChatApp chọn TCP: ưu tiên đủ byte và đúng thứ tự cho chat văn bản và file.

# TCP chuyển byte. Ứng dụng quy định cách trao đổi.

Cùng một nền TCP, có thể xây các kiểu ứng dụng khác nhau. Ví dụ khi gửi “Xin chào”:

CHATAPP DESKTOP HIỆN TẠI

## Packet riêng

Client Java tạo MSG_DIRECT; server hiểu quy tắc Packet do nhóm tự xây.

`Packet của ChatApp`

`TCP → IP → mạng`

NẾU XÂY WEB API

## HTTP

Client gửi yêu cầu, ví dụ POST /messages; server trả phản hồi. Realtime có thể thêm polling hoặc SSE.

`HTTP/1.1 hoặc HTTP/2`

`TCP → IP → mạng`

NẾU XÂY WEB CHAT REALTIME

## WebSocket

Giữ kết nối để hai phía chủ động gửi thông điệp; thuận tiện cho chat trên trình duyệt.

`WebSocket thông dụng`

`TCP → IP → mạng`

ChatApp hiện dùng cột trái. HTTPS/WSS bổ sung TLS; HTTP/3 dùng QUIC trên UDP.

# Packet là vỏ gói. Payload chứa dữ liệu thao tác.

## 01 · Packet: đủ 5 trường

```
{
  "type": "MSG_DIRECT",
  "sender": "giahuy",
  "recipient": "sonlam",
  "payload": "(chuỗi JSON bên phải)",
  "timestamp": 1792200000000
}
```

type: thao tác · sender: người gửi · recipient: đích nhận · payload: dữ liệu · timestamp: mili giây Unix.

## 02 · Payload khi MSG_DIRECT

```
{
  "messageId": "m-01",
  "sender": "giahuy",
  "senderFullName": "Trần Gia Huy",
  "recipient": "sonlam",
  "content": "v1:…bản mã…",
  "isGroup": false,
  "timestamp": 1792200000000,
  "status": "SENT",
  "isEncrypted": true
}
```

Payload là chuỗi JSON của ChatMessage. messageId ghép receipt với tin; content mã hóa, isEncrypted đánh dấu.

```
AUTH_* · USER_* · ROOM_* · MSG_* · HISTORY_* · FILE_* · SYSTEM_ERROR
```

Minh họa trước lớp bọc TCP; content là bản mã ví dụ. Trên dây: Packet JSON → AES-GCM → Base64 + newline.

# Đăng nhập: kiểm tra qua hai tầng.

| THỰC HIỆN | XỬ LÝ | KẾT QUẢ |
|---|---|---|
| 01 · Giao diện | Kiểm tra ô trống, đọc host và cổng. | Mở kết nối TCP tới server. |
| 02 · ChatClient | Gửi AUTH_LOGIN kèm username, password. | ClientHandler đọc JSON theo dòng. |
| 03 · Server + DB | Đối chiếu hash + salt; kiểm tra tài khoản online. | Hợp lệ → ghi nhận phiên, vào lobby. |
| 04 · Client nhận | Nhận AUTH_LOGIN_SUCCESS qua listener. | Đóng form đăng nhập, mở cửa sổ chat. |

> NHÁNH THẤT BẠI — Sai thông tin / đăng nhập trùng → AUTH_LOGIN_FAIL. Không kết nối được → báo lỗi tại form.

Server nhận Socket rồi giao ClientHandler cho cached thread pool; client có luồng nhận riêng, Swing cập nhật UI qua EDT.

# Một tin nhắn đi từ A tới B ra sao?

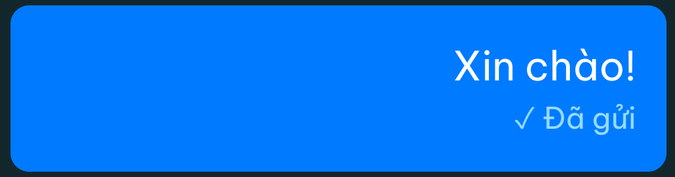
| THỰC HIỆN | XỬ LÝ | KẾT QUẢ |
| --- | --- | --- |
| 01 · Giao diện A | Chọn B, nhập nội dung, bấm Gửi; bỏ qua chuỗi rỗng. | Gọi ChatClient.sendDirectMessage(). |
| 02 · ChatClient A | Mã hóa riêng thân tin bằng AES-256-GCM; giữ bản hiển thị plaintext, tạo Packet MSG_DIRECT. | Ghi một dòng JSON lên TCP; flush(). |
| 03 · Server | ClientHandler chuẩn hóa sender, xác thực bản mã; tìm B trong onlineClients. | Chuyển gói tin qua kết nối TCP của B. |
| 04 · Client B | Luồng nhận giải mã; gọi onMessageReceived(). | Swing cập nhật hội thoại và bong bóng tin. |

| NGƯỜI NHẬN OFFLINE | sendDirectPacket() trả false; hiện chưa có hàng đợi lưu tin để chuyển khi B online lại. |
| --- | --- |

Server lưu bản mã trong SQLite; client tải lịch sử bằng HISTORY_REQUEST / HISTORY_RESPONSE và giải mã để hiển thị.

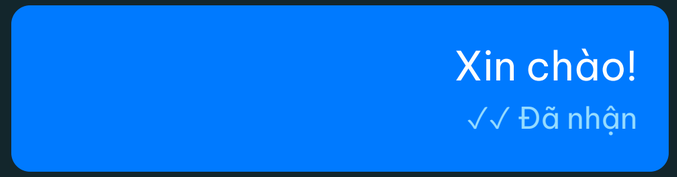
# UI hiện tại hiển thị trạng thái tin riêng thế nào?

SENT · PHÍA NGƯỜI GỬI A



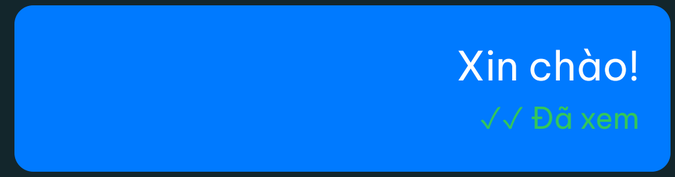

A gửi qua kết nối rồi thêm bong bóng cục bộ. Chưa có xác nhận B nhận tin hoặc server lưu thành công.

DELIVERED · CLIENT B ĐÃ NHẬN




B giải mã MSG_DIRECT rồi gửi MSG_DELIVERED. Server chuyển receipt cùng messageId về A.

READ · HỘI THOẠI ĐANG HIỂN THỊ




Cửa sổ B có focus và đang chọn chat với A: gửi MSG_READ khi nhận tin hoặc khi mở lại hội thoại.

**Phạm vi UI:** nhãn chỉ hiện dưới tin riêng do mình gửi; chat nhóm không có nhãn này. “Đã xem” là trạng thái giao diện, không chứng minh người dùng đã đọc nội dung.

**Giới hạn:** trạng thái tăng dần, không lùi READ về DELIVERED. Receipt được định tuyến trong phiên; chưa lưu cập nhật trạng thái vào SQLite để khôi phục đầy đủ sau đăng nhập lại.

Minh họa nhãn theo ChatBubblePanel; điều kiện theo MainChatWindow và ChatClient. Không phải ảnh chụp phiên chạy thực tế.

# Phòng chat quyết định ai nhận tin.

| THỰC HIỆN | XỬ LÝ | KẾT QUẢ |
| --- | --- | --- |
| 01 · Người tạo | Nhập tên phòng, gửi ROOM_CREATE. | RoomManager tạo phòng và thêm chủ phòng. |
| 02 · Server | Trả ROOM_CREATE_SUCCESS; phát ROOM_LIST_UPDATE. | Các client cập nhật danh sách phòng. |
| 03 · Thành viên | Gửi ROOM_JOIN rồi MSG_GROUP với roomId. | Server lấy danh sách thành viên phòng. |
| 04 · Server → Client | Gửi tới thành viên online, trừ người gửi. | Client giải mã và lưu theo roomId. |

| | |
| --- | --- |
| RỜI PHÒNG | ROOM_LEAVE cập nhật thành viên; server phát lại danh sách phòng. Sảnh lobby không cho rời. |

Trạng thái đang nhập là thông báo USER_TYPING; tin nhóm không có receipt đã nhận/đã xem theo từng thành viên.

# Chỉ truyền khi người nhận đồng ý.

| THỰC HIỆN | XỬ LÝ | KẾT QUẢ |
|---|---|---|
| 01 · Client A | Chọn tệp hợp lệ trong chat 1–1; lưu tệp chờ gửi. | FILE_REQUEST → Server → B. |
| 02 · Client B | Xem tên, dung lượng; chọn nhận và thư mục lưu. | P2PFileReceiver mở ServerSocket(0). |
| 03 · B → Server | Gửi FILE_ACCEPT kèm cổng nhận. | Server bổ sung IP nếu thiếu, chuyển về A. |
| 04 · Client A | Lấy lại tệp chờ gửi; tạo P2PFileSender. | Kết nối TCP trực tiếp tới IP:cổng của B. |

> NẾU TỪ CHỐI — B gửi FILE_REJECT qua server. A xóa tệp khỏi danh sách chờ và hiển thị thông báo.

# Socket riêng xử lý toàn bộ byte tệp.

| THỰC HIỆN | XỬ LÝ | KẾT QUẢ |
|---|---|---|
| 01 · B chờ A | accept() trên cổng P2P, timeout chờ 30 giây. | Kết nối thành công → trao đổi offset. |
| 02 · A gửi | Đọc tối đa 64 KB từ tệp, ghi vào socket. | Lặp đến đủ byte hoặc bị hủy / lỗi. |
| 03 · B ghi | Đọc socket, ghi nối tiếp vào .part qua buffer. | Cập nhật % và tốc độ khoảng mỗi 0,5 giây. |
| 04 · Hoàn tất | B đóng tệp, kiểm tra SHA-256 rồi đổi tên .part. | B gửi xác nhận; A nhận xác nhận rồi báo hoàn tất. |

> NHẬN THIẾU BYTE — B giữ .part khi ngắt. A chỉ báo thành công sau xác nhận kiểm tra và lưu tệp từ B.

Đọc → gửi → ghi theo buffer 64 KiB, tránh nạp toàn tệp vào RAM; không suy ra mức RAM hay tốc độ khi chưa đo.

# Tiếp tục cần một yêu cầu gửi mới.

| THỰC HIỆN | XỬ LÝ | KẾT QUẢ |
| --- | --- | --- |
| 01 · Ngắt / hủy | B giữ dữ liệu nhận dở trong tên-tệp.part. | Không tự kết nối lại trong nền. |
| 02 · Gửi lại | A chọn lại cùng tệp; B nhận vào cùng thư mục. | Lặp lại FILE_REQUEST / FILE_ACCEPT. |
| 03 · Trao đổi offset | B gửi độ dài .part bằng writeLong(offset). | A dùng RandomAccessFile.seek(offset). |
| 04 · Truyền tiếp | B mở .part ở chế độ append; A gửi phần còn lại. | Nhận đủ byte → kiểm tra SHA-256 → lưu tệp đích. |

**ĐIỀU KIỆN** Dùng lại .part gắn SHA-256 trong cùng thư mục. Nhận xong phải kiểm tra hash toàn tệp trước khi hoàn tất.

# Nêu đúng những gì đã triển khai.

MẬT KHẨU

## SHA-256 + salt

Salt ngẫu nhiên 16 byte; dữ liệu tài khoản lưu trong SQLite.

TIN NHẮN

## AES-256-GCM

Từng tin nhắn mới mã hóa riêng; khóa Base64 32 byte cấu hình bên ngoài, dùng chung cho server và client.

GIỚI HẠN

## Giới hạn khóa chung

Không phải mã hóa đầu cuối. Mất khóa thì lịch sử mới không đọc được; lịch sử plaintext cũ vẫn giữ nguyên.

Khóa tin nhắn: CHATAPP_TEXT_KEY (Base64 32 byte). Lớp bọc TCP và P2P file vẫn dùng cơ chế AES demo hiện có.

# Khi luồng chính không đi đến cuối.

| THỰC HIỆN | XỬ LÝ | KẾT QUẢ |
| --- | --- | --- |
| Không vào được app | Server chưa chạy / sai host-cổng / sai tài khoản. | Báo tại form; người dùng sửa và thử lại. |
| Không bắt đầu gửi tệp | Chưa chọn người/phòng, tệp rỗng hoặc người nhận từ chối. | Chặn thao tác hoặc thông báo từ chối. |
| P2P bị gián đoạn | Lỗi socket, timeout chờ kết nối hoặc bấm hủy. | Dialog báo lỗi; dùng lại .part nếu có. |
| Mất kênh chat | Client phát hiện kết nối server đã mất. | Báo lỗi và thoát ứng dụng hiện tại. |

GIỚI HẠN CẦN NÊU

Thiếu/sai khóa phải sửa cấu hình. Chưa có tự reconnect hay hàng đợi giao tin offline; file có kiểm tra SHA-256 trước khi hoàn tất.

# Một kịch bản, đi qua toàn hệ thống.

01

## Kết nối

Chạy server và hai client; đăng nhập tài khoản.

02

## Trò chuyện

Gửi tin cá nhân, tạo phòng và chat nhóm.

03

## Chia sẻ

Gửi tệp, ngắt giữa chừng và thử truyền tiếp.

> Kiểm chứng đã có: 23 test qua ở lần build trước · crypto · chat/lịch sử · file P2P

Demo UI và hai máy LAN vẫn cần chạy thực tế; bộ test không thay thế quan sát thao tác trên giao diện.

CHATAPP

# Cảm ơn thầy cô và các bạn!

TCP Socket · Chat riêng/nhóm · File P2P · Mã hóa tin nhắn.

Nhóm xin nhận câu hỏi và góp ý.